

---

# Can This Agent Even Do That?

## A Decidable Goal-Achievability Type Discipline for LLM-Synthesized Agent Skills

---

Anonymous Author(s)

Affiliation

Address

email

### Abstract

1 LLM agents are increasingly driven by natural-language specifications, expressed  
2 through artifacts such as `SKILL.md` and `AGENT.md` files that describe an agent’s  
3 capabilities, tools, and workflows. Agent Skills, for example, package instructions  
4 that guide an agent in performing specific tasks [5]. But how can we know whether  
5 such specifications are themselves correct? Even if an LLM agent follows a speci-  
6 fication exactly, can it actually achieve the goal that the specification is intended to  
7 accomplish? Or might the goal be fundamentally unachievable, discovered only  
8 after spending substantial time and tokens executing the agent—or even ending  
9 in deadlock? We present a *skill compiler* that formally validates, before runtime  
10 execution and without relying on an LLM judge, whether a skill’s *goal is achiev-*  
11 *able*. The compiler combines capability-guarded reachability from automated  
12 planning with deadlock-freedom from **simple multiparty sessions (SMPS)**, decided  
13 *directly* over session configurations and a goal-marked global type synthesized  
14 from the natural-language specification. Our approach follows the **SMPS** discipline  
15 of checking a session directly against a global protocol [1, 2]. The approach begins  
16 by using an LLM to extract the content of a natural-language specification and  
17 compile it into a formal logical pack consisting of preconditions, effects, and goals,  
18 which are then consumed by the inference rules. The checker is specified by a  
19 Coq-verified, refutation-sound abstraction theorem. We further establish subject  
20 reduction and session fidelity for the direct discipline. The single-label commu-  
21 nication fragment with bystander interleaving is mechanized, while world-changing  
22 action interleaving is proved under explicit hypotheses of goal stability and effect  
23 commutativity. We formally define the language and its behavior, specify how  
24 skills and sessions are checked, and prove that the resulting checking framework  
25 has several important correctness properties. We also identify which part of the  
26 system remains decidable and show where the boundary of undecidability begins  
27 when participants can be added dynamically.

## 28 1 Introduction

29 A modern agent is no longer merely a chatbot, but an autonomous system capable of taking actions  
30 beyond text generation—for example, writing code, sending emails, or executing trades. Such  
31 behaviour is often guided by natural-language artifacts that specify instructions, workflows, and  
32 constraints for the agent [3, 4, 6, 7]. A *skill* is one common way of expressing such guidance. It is a  
33 folder containing instructions and tooling that an agent can discover and load as needed [5], thereby  
34 *harnessing* a general-purpose agent toward a particular outcome.

35 There is growing evidence that this guidance matters. Across seven state-of-the-art open-source  
 36 multi-agent systems, failure rates range from 41% to 86.7% [8]. More importantly, some of these  
 37 failures can be addressed without changing the underlying model. In one system, simply adding a  
 38 task-objective verification step to the specification improves task success by 15.6% [8, Appendix H].  
 39 Likewise, repairing the harness based on execution traces improves task completion by 6.3 to 18.4  
 40 percentage points across four agent benchmarks [4]. These results show that the instructions and  
 41 structure surrounding an agent can matter as much as the model itself.

42 The question, however, is how we can know whether such guidance is sufficient to achieve the  
 43 outcome it declares. A plan may *book a flight and email a confirmation* even though no email tool is  
 44 available. It may pursue *a flight under \$500* even though the available booking tool can return only  
 45 premium fares. Or a planner may wait for a worker’s result while the worker waits for the planner’s  
 46 answer, with each expecting a message that the other will never send. These are not merely execution  
 47 failures. In each case, the goal is already unreachable before execution begins.

48 Such failures are structural and recurring, yet today the only way to discover them is often to run  
 49 the skill and inspect what went wrong afterwards [4]. By then, the agent may already have spent  
 50 substantial time and tokens taking actions, and a multi-agent system may already have become stuck  
 51 waiting for communication that will never occur. A simple static check could, in principle, identify  
 52 these problems before execution.

53 This paper asks a deliberately narrow question and answers it with a deterministic formal typing  
 54 system: **given an LLM skill, can its goal be achieved at all?** We do not, here, verify that every step  
 55 is safe, nor that the agent behaves correctly on every possible input. Instead, we decide *achievability*.  
 56 The analysis is tolerant: small details, detours, and payload variation should not trigger a false alarm.  
 57 At the same time, it is *sound for refutation*: when the compiler concludes that a goal is impossible,  
 58 that verdict is correct.

## 59 1.1 The idea, and the trust boundary

60 The architecture, shown in Figure 1, has two parts separated by a trust boundary. First, an LLM  
 61 performs *compaction*: it reads the natural-language skill and turns it into a formal *pack* containing  
 62 capabilities with preconditions and effects, a goal-marked global protocol, and an initial state. This  
 63 step is *untrusted*: the LLM may misunderstand the skill or omit relevant details.

64 A deterministic *checker* then decides achievability using only the resulting pack. The checker is  
 65 mechanically proved sound: if it returns *impossible*, then the goal is impossible under the formal pack  
 66 it received. If the LLM omits a constraint or capability from the original skill, the checker cannot  
 67 recover information that is not present in the pack; it will instead reason about the incomplete pack.  
 68 Conversely, if the LLM introduces a constraint that was not in the original skill, the checker may  
 69 report the resulting goal as impossible. Thus, our guarantee is about the checker’s verdict with respect  
 70 to the formal pack, not about whether the pack perfectly represents the original natural-language skill.

71 This does not make the compaction step a weakness that we simply ignore. The formal pack provides  
 72 a structured target for the LLM to extract. Rather than asking the LLM to freely invent a formal  
 73 model, we give it a predefined logical structure: capabilities have preconditions and effects, goals are  
 74 explicit, and the protocol follows a fixed syntax. This makes compaction a constrained translation  
 75 task and gives us a concrete artifact that can be inspected before execution.

76 Crucially, the gap between *what the user meant* and *what was formalized* cannot be eliminated. We  
 77 instead make this gap explicit at the compaction step, where the resulting pack can be inspected and  
 78 checked before execution. When the compaction is accurate, the checker can identify goals that are  
 79 impossible before the agent spends time and tokens executing the skill. When the compaction is  
 80 inaccurate, the checker still provides a sound verdict for the formal pack it received. The key design  
 81 idea is therefore to move achievability checking from runtime execution to an explicit, inspectable  
 82 checkpoint before execution.

## 83 1.2 Contributions

- 84 1. **A goal-achievability type discipline** (Section 3 and Appendices A–C) that combines  
 85 capability-guarded reachability from automated planning with deadlock-freedom from  
 86 multiparty session types. The discipline checks whole session configurations directly against

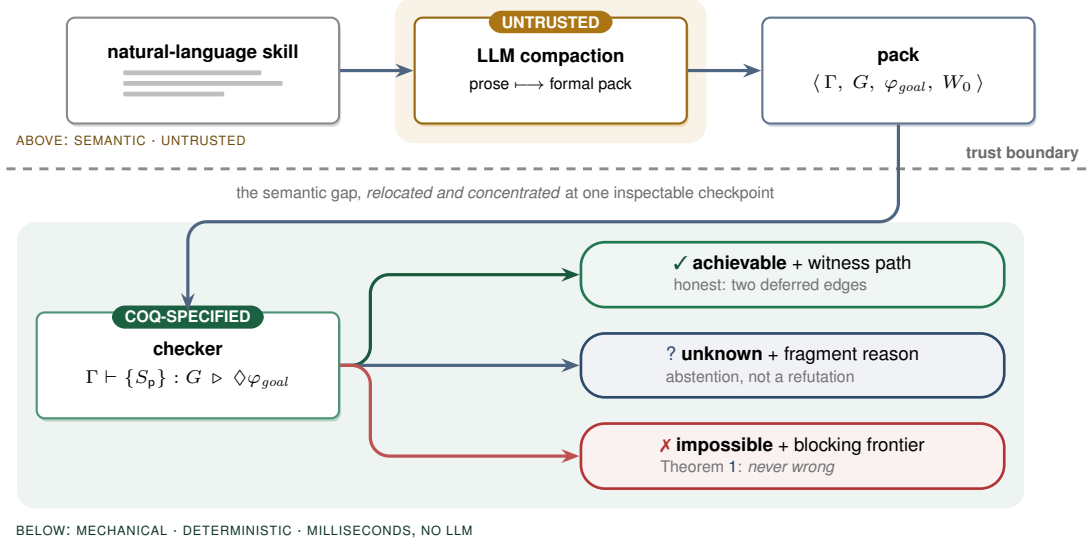


Figure 1: The trust boundary. An untrusted LLM compacts natural-language instructions into a formal pack. A deterministic checker then decides achievability over the pack using a Coq-verified abstraction theorem. The verdicts are asymmetric: *impossible* is sound (Theorem 1), while *achievable* depends on the accuracy of the pack (Section 3.1).

87 a goal-marked global type synthesized from the skill. Building on the multiparty session  
 88 type tradition of Honda, Yoshida, and Carbone [12, 13], and the direct session-checking  
 89 approach of **SMPS** [1, 2], it adds capabilities, explicit goals, and a refutation-sound but  
 90 achievement-incomplete analysis for LLM-drafted skills.

91 2. **A mechanized soundness specification** (Appendix D) formalized in Coq. We prove a  
 92 refutation-sound abstraction theorem parameterized by simulation hypotheses, together  
 93 with tolerance soundness and capability monotonicity: adding capabilities cannot make an  
 94 achievable goal impossible. We also establish operational correspondence through subject  
 95 reduction and session fidelity for the direct judgment over a labelled transition system. For  
 96 single-label communication, we mechanize the cases where other participants continue to  
 97 act concurrently; the world-changing action cases are proved on paper under explicit side  
 98 conditions.

99 3. **A decidability boundary** (Appendix D). Achievability is decidable when the set of par-  
 100 ticipants is fixed. We also show that an extension allowing the dynamic addition of an  
 101 unbounded number of participants becomes undecidable.

102 4. **An implementation and evaluation** (Sections 4–5). We implement the checker together  
 103 with an LLM-compaction front-end and a schema gate, and evaluate it on a corpus covering  
 104 the main failure modes. The results are consistent with the theoretical guarantees: we  
 105 observe no false impossibility verdicts, while every false achievability verdict falls into one  
 106 of the two deferred cases.

## 107 2 Overview by Example

108 Consider a one-agent skill whose goal is *a flight is booked and a confirmation is sent*. The author  
 109 writes prose; the LLM compacts it. Two compactations differ only in whether an email tool is in scope.

$$\varphi_{goal} = \text{booked} \wedge \text{confirmation\_sent}$$

| capability                                | precondition | effect                      |
|-------------------------------------------|--------------|-----------------------------|
| <i>variant A</i> — no email tool in scope |              |                             |
| search                                    | $\top$       | $Add\{searched\}$           |
| filter                                    | searched     | $Add\{filtered\}$           |
| book                                      | filtered     | $Add\{booked\}$             |
| <i>variant B</i> — variant A, plus:       |              |                             |
| email                                     | booked       | $Add\{confirmation\_sent\}$ |

Under STRIPS *frame semantics* — a predicate is false unless an effect makes it true — variant A has no action establishing `confirmation_sent`, so no reachable state satisfies the goal: the checker returns *impossible* and points at the missing establisher. Variant B reaches the goal along `search · filter · book · email`, returned as a witness. The same example is mechanized in Appendix D (the concrete instance `FlightInstance`): variant A is *provably* unachievable, yet a booking is still reachable — the refutation is about the missing tool, not a dead protocol.

Tolerance is essential, and is where a naive reachability check would misfire. Extra status messages, a clarifying detour, or payload variation must not refute the goal. We obtain this from three orthogonal relaxations made precise in Appendices A–C: *existential may-reachability* (a single good path suffices), *session subtyping* (interface slack in the safe direction, folded directly into the single conformance rule rather than a separate relation), and *goal-relevant abstraction* (payload detail collapsed before checking).

### 3 Verification Model and Signals

**Declared pack.** The checker consumes  $P = \langle \Gamma, G, \varphi_{goal}, W_0 \rangle$ : capability declarations  $\Gamma$  with guards and STRIPS-style effects; a goal-marked global protocol  $G$  (formally, a *global type*; Appendix A); a logical goal  $\varphi_{goal}$ ; and an initial abstract world  $W_0$ . The LLM that compacts prose into  $P$  is outside the trusted core. A deterministic schema gate first enforces the decidable fragment: finite static roles, guarded recursion, finite-domain booleans, and quantifier-free linear integer arithmetic. The formal syntax and labelled semantics are given in Appendices A–B.

**Heterogeneous verification signals.** Achievability is not reduced to a learned score. The judgment composes signals that live on different substrates:

| Signal                                                  | Formal role                                                      | Evidence returned                                                                |
|---------------------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------|
| tool availability and effects<br>coordination structure | capability guards; world transition<br>direct global conformance | missing capability or witness action<br>offending handoff or conformant protocol |
| goal and cost refinements                               | logical reachability; QF-LIA                                     | unsatisfied conjunct, blocked guard, or model                                    |
| human intent review<br>runtime payload checks           | validates prose-to-pack fidelity<br>deferred monitor obligation  | accepted or revised declared goal<br>blame-bearing execution event               |

The first three signals are composed before execution. The latter two are explicit obligations rather than assumptions silently folded into the static verdict.

**Decision procedure.** After the schema gate, the checker (i) rejects references to undeclared capabilities and goal atoms with no establisher; (ii) checks declared role behaviors against  $G$ , requiring exact sender choices while allowing receiver-side branch supersets; and (iii) explores existential protocol/world successors from  $W_0$ , using Z3 for guards and refinements. An unsatisfied goal marker is a blocking checkpoint. Solver timeouts and packs outside the static fragment cannot justify refutation. The interface is therefore tri-valued:

|                      |                                                     |
|----------------------|-----------------------------------------------------|
| $IMPOSSIBLE(F)$      | a sound blocking frontier $F$ ,                     |
| $ACHIEVABLE(\pi, O)$ | a witness path $\pi$ and deferred obligations $O$ , |
| $UNKNOWN(r)$         | an abstention with fragment reason $r$ .            |

Verdicts also carry the pack digest, checker semantics, and artifact version so that feedback remains attributable to the exact declaration checked.

143 **Guarantees.** Let  $abs$  map concrete protocol/world configurations to abstract configurations, and  
 144 assume step and goal simulation for the declared pack (Appendix D). If no abstract run reaches  
 145  $\varphi_{goal}$ , no concrete run represented by that pack reaches the concrete goal (Theorem 1). Coarsening  
 146  $\alpha$  and adding capabilities cannot create a false refutation (Theorems 2–3). Direct conformance has  
 147 subject-reduction and session-fidelity results (Theorems 4–5); the Coq artifact covers single-label  
 148 communication with bystander interleaving, while the paper states explicit stability and commutativity  
 149 assumptions for world-changing interleavings. The static fragment is decidable; unbounded dynamic  
 150 spawning is an explicit undecidable extension, for which the implementation abstains. Full rules,  
 151 assumptions, proofs, and mechanization scope appear in Appendix D.

### 152 3.1 Structured feedback and human oversight

153 The blocking frontier and witness are inspectable, heterogeneous feedback rather than a scalar reward.  
 154 A bounded repair adapter may ask the untrusted LLM to fix a non-projectable protocol, but it may  
 155 neither invent capabilities nor weaken the goal, and every revision is rechecked deterministically.  
 156 This realizes a small agent-verifies-agent loop while keeping the human checkpoint at the irreducible  
 157 intent-fidelity stage. Automated prompt or scaffold search is an application of these verdicts, not an  
 158 empirical contribution claimed here.

## 159 4 Implementation

160 The executable decision path is approximately 1.45K lines of Python, including the checker, pack gate,  
 161 formula layer, and session adapter. The compaction front-end issues the artifact prompt to an LLM  
 162 and passes the result through a deterministic *schema gate* before the checker sees it, so malformed  
 163 packs are rejected without crashing the trusted core. The Coq artifact contains three developments and  
 164 two assumption-audit harnesses: reachability soundness core (Theorems 1–3 plus `FlightInstance`,  
 165  $\sim 230$  lines); the direct-typing core (the head-move action safety plus `HandoffInstance`,  $\sim 310$   
 166 lines); and the operational-correspondence core (Theorems 4–5 for the single-label communication  
 167 fragment with full interleaving,  $\sim 250$  lines); all three are checked in CI under pinned Coq 8.18.0  
 168 with no axioms, verified by `Print Assumptions`. The checker implements the corresponding  
 169 abstractions from Appendices A–C: STRIPS frame semantics for the world, existential search over  
 170 the protocol control structure, and Z3 for guard/goal satisfiability and arithmetic refinements. On  
 171 success it returns the witness path; on refutation, the blocking frontier (missing capability, unsatisfiable  
 172 guard, unestablished goal conjunct, or non-projectable handoff). The optional LLM front-end may  
 173 use a `NON_PROJECTABLE` counterexample for one bounded, structure-only repair round; it may not  
 174 invent capabilities or weaken the goal, and the deterministic checker remains the final referee. This is  
 175 a small reflective-verification loop, not a general scaffold-search result.

176 The conformance step ( $G \vdash (\mathbb{M}; W)$ , Section C.2) is **implemented by means of a simple type**  
 177 ***inference algorithm since all typing rules are structural in  $(\mathbb{M}; W)$ . At each step we freely choose***  
 178 ***either one or two participant processes to reduce or a goal satisfied by the world obtaining the set of***  
 179 ***sessions which must be typed as premises of the typing rule.***

180 The adapter also discharges the participant side conditions of T-COMM/T-ACT/T-GOAL: it computes  
 181  $\text{prt}(G)$  and compares it with the roles the pack declares behaviour for. One direction refutes — a  
 182 declared role outside  $\text{prt}(G)$  has contract End, which nothing non-trivial refines. The other cannot: a  
 183 participant the pack declares nothing for offers no behaviour to refute, yet the judgment of Section C.2  
 184 ranges over a whole session with  $\text{prt}(G) = \text{prt}(\mathbb{M})$ , so the checker returns those roles alongside the  
 185 verdict rather than leaving the assumption implicit.

## 186 5 Evaluation

187 We assembled a corpus of 15 skills spanning the canonical failure modes — hallucinated planning,  
 188 retry-forever loops, coordination deadlock, refinement violations — with achievable controls (in-  
 189 cluding detours and informed branches) and two deliberately *spurious* cases exercising the deferred  
 190 edges. Each skill carries a ground-truth semantic label; the positive class is *achievable*. Evaluation is  
 191 deterministic and CPU-only: it requires no training or live model inference, completes the 15-case  
 192 matrix in under one second on a commodity laptop, and gives each individual Z3 query a 10-second  
 193 timeout.

|                  | predicted achievable | predicted impossible |
|------------------|----------------------|----------------------|
| truly achievable | TP = 6               | FN = 0               |
| truly impossible | FP = 2               | TN = 7               |

Two claims, both confirmed. **Soundness:** FN = 0 — no achievable goal was wrongly refuted, the empirical face of Theorem 1 (achievability recall  $6/6 = 100\%$ ). **Incompleteness, contained:** FP = 2, and both are exactly the planted spurious cases — one a false payload post-condition (Layer-C obligation), one an under-specified goal (intent edge). No structural failure was missed in this proof-of-concept corpus; this is evidence of coverage, not a proof that the two residues exhaust all empirical failure modes. Each refutation reason fires on its matching mode:

| Failure mode (source)                                      | Verdict reason     |
|------------------------------------------------------------|--------------------|
| hallucinated planning — invokes a non-existent tool        | MISSING_CAPABILITY |
| hallucinated planning — no establisher for a goal conjunct | GOAL_UNSAT         |
| retry-forever loop — guard never satisfiable               | BLOCKED_GUARD      |
| coordination deadlock — unobserved choice / bad handoff    | NON_PROJECTABLE    |
| refinement violation — budget unsatisfiable on every run   | GOAL_UNSAT         |

**What the check costs.** With no model in the trusted path, the checker and the deterministic front-end spend *zero* tokens; only compaction spends any, once per skill *version* and linearly in the skill’s length, against a run it avoids that recurs per *invocation* and is quadratic in its turns. Under an explicit, conservative model of that run (Appendix E), compacting the 15-spec corpus costs 12.2K tokens and avoids  $\sim 1.07\text{M}$  per round of invocations —  $\sim 87\times$  leverage, unbounded deterministically — so break-even falls inside the first prevented run in every failure mode, while a skill that is *not* broken pays 2.9% of one successful run for the check, or nothing.

A separate six-case fragment suite exercises behavior absent from the headline matrix: tail-recursive retry and non-terminating control, dynamic spawning, protocol-independent refutation despite spawning, and conformant versus non-conformant declared role behavior. It yields two ACHIEVABLE, three IMPOSSIBLE, and one UNKNOWN. The abstention is reported separately rather than counted as a false negative, because UNKNOWN makes no refutation claim. Together the suites contain 21 declared packs; the 15-case matrix remains the only binary ground-truth matrix.

## 6 Related Work

**Prior-art note.** The claims below reflect the authors’ survey at submission time; the fusion’s novelty should be confirmed against a systematic search of the planning-meets-behavioural-types literature, which is dispersed across communities.

*Multiparty session types* [13, 16, 9] supply deadlock-freedom via projection and subtyping [14, 10]; our calculus and directed-choice global types follow the modern synchronous formulation of the Yoshida line [9–11]. We adopt the synthetic line [1, 2]: Section C.2 checks conformance directly against a whole session configuration rather than projected local types. Our direct rule corresponds to the conservative, plain-merge condition that bystanders behave uniformly across an unobserved choice; it does not subsume every protocol accepted by modern full merge. The new contribution is the world and goal layer: capability guards, effects, goal observation, and the asymmetric refutation guarantee are not modelled by classical MPST or new SMPS. *Automated planning* [17] supplies capability-guarded goal reachability; we reuse its frame semantics but embed it in a multiparty choreography. The contribution is the *fusion*, decided over an goal-marked global type synthesized by an LLM. Recent agent-verification work is adjacent but differently aimed: *provable coordination via message sequence charts* [18] projects a global choreography for LLM agents but targets coordination correctness, not goal achievability; its runtime-planning extension also establishes that LLM synthesis of a coordination protocol is itself prior art. *VeriGuard* [19] separates offline policy verification from online monitoring as a safety shield; *agent behavioural contracts* [20] provide probabilistic, runtime-enforced compliance and composition conditions for multi-agent chains, whereas our verdict is static and sound for refutation relative to the pack.

*Skill-bundle scanners and verified-skill catalogs* are complementary. NVIDIA’s SkillSpector [21] and Verified Agent Skills pipeline [22] address the same deployment object—portable agent skills—but



ask a different question. SkillSpector is a pre-publication security control: it normalizes a skill bundle and runs deterministic scanners such as static pattern detectors, AST and taint analyzers, manifest-consistency checks, metadata-poisoning checks, supply-chain checks, and optional LLM-backed semantic analyzers. This is valuable admission control for skills, but it is not a compiler or type discipline in the sense used here. As published, SkillSpector does not aim to provide an operational semantics of goal-marked skills, a goal-reachability decision, or a refutation-soundness theorem. A SkillSpector-like pass belongs at the boundary of the compiler: before or alongside LLM compaction, it can vet the incoming SKILL.md, code, dependencies, and MCP metadata; after compaction, it can help justify the honesty of declared capabilities and least-privilege metadata. The type checker then consumes the sanitized formal pack and proves the narrower claim that the declared goal is not structurally impossible.

None of the above frames the static check as goal achievability over a declared world and goal-marked protocol, nor gives the same refutation-sound/achievement-incomplete asymmetry.

## 7 Limitations and Future Work

- **Relative soundness.** Everything is proved about the declared  $\Gamma$  and  $S$ . If the prose lies (a “read-only” tool that writes), the checker verifies a fiction; honest-declaration is a separate obligation, of which this makes only the interaction slice decidable. A practical compiler should therefore include a skill-bundle security pass—for example a SkillSpector-like scanner—to inspect SKILL.md, scripts, dependencies, manifests, and permission metadata before the formal pack is trusted by the achievability checker.
- **Static topology for totality.** Dynamic spawning drops the procedure to a semi-decision (Proposition 2); a practical system answers UNKNOWN rather than guessing.
- **Trusted abstraction.** A coarse  $\alpha$  silently widens tolerance (more false achievable), never breaking Theorem 1 but weakening usefulness.
- **Engineering gaps.** The mechanization covers the generic refutation-sound abstraction theorem and concrete examples, head-move action safety, and subject reduction/session fidelity with full bystander interleaving for the single-label communication model, all axiom-free. The executable checker is schema-gated and tested against that specification, but the STEP-SIM/GOAL-SIM instantiation for its exact symbolic state is not mechanized. The Coq correspondence model also does not yet combine observable goal labels, participant matching, multi-branch choice, and world-changing interleaving in one development. Still open are that faithful mechanization; a head-normalization/permutation bridge from interleaved session runs to the head-ordered reachability search; equivalence between the declarative judgment and the projection-based adapter; and a concrete-session realization theorem for abstract witnesses. The 21-pack evaluation remains a proof of concept.

**Broader impact.** Pre-execution refutation reduces wasted computation (Appendix E) and prevents structurally impossible agent plans from reaching deployment. The same formal verdict can, however, create false confidence when an inaccurate pack omits a harmful tool effect or weakens the user’s actual goal. We therefore expose the declared pack, witness or blocking frontier, and deferred obligations rather than presenting a single trust score. Intent review and runtime payload monitoring remain necessary before the method is used in consequential settings; this work does not evaluate fairness, privacy, or deployment safety.

## 8 Conclusion

An LLM can draft an agent’s plan from intent; the open question has been whether anything can tell, cheaply and before execution, that the plan is doomed. For the *achievability* question the answer is yes, relative to a declared pack: combine planning’s capability-guarded reachability with a synthetic, directly-typed global protocol; treat LLM compaction as untrusted; and use a refutation-sound abstraction so an *impossible* verdict is never wrong about that declared model. *Achievable* remains honest about payload and intent fidelity, while *Unknown* abstains outside static topology. This yields a structured, inspectable verification signal for agent skills and a path toward, rather than a completed proof of, stronger per-step safety.

## References

- [1] Franco Barbanera, Mariangiola Dezani-Ciancaglini & Ugo de'Liguoro (2024): *Un-projectable Global Types for Multiparty Sessions*. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella & Matteo Rossi, editors: *PPDP*, ACM Press, pp. 15:1–15:13,
- [2] Francesco Dagnino, Paola Giannini & Mariangiola Dezani-Ciancaglini (2023): *Deconfined Global Types for Asynchronous Sessions*. *Logical Methods in Computer Science* 19(1), pp. 1–41,
- [3] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, X. Liao, Y. Ji, J. Wang, J. Gu, Y. Ge, W. Xu, X. Fang, X. Xu, T. Zhao, Y. Kim, T. Wang, J. Hamm, S. Krishnaswamy, J. Huan, C. Reddy. *Agent Harness Engineering: A Survey*. 2026.
- [4] M. Chen, J. Wang, Z. Liu, Y. Wang, H. Zheng, Q. Wang. *From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws*. arXiv:2606.06324, 2026.
- [5] Agent Skills Standard. Specification. <https://github.com/agentskills/agentskills/blob/main/docs/specification.mdx>, 2025.
- [6] S. Hu, C. Lu, J. Clune. *Automated Design of Agentic Systems*. *ICLR*, 2025.
- [7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. *AFlow: Automating Agentic Workflow Generation*. *ICLR*, 2025.
- [8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica. *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657, 2025.
- [9] N. Yoshida, L. Gheri. *A Very Gentle Introduction to Multiparty Session Types*. *ICDCIT*, LNCS 11969, 2020.
- [10] S. Ghilezan, S. Jakšić, J. Pantović, A. Scalas, N. Yoshida. *Precise Subtyping for Synchronous Multiparty Sessions*. *J. Log. Algebr. Meth. Program.* 104, 2019.
- [11] A. Scalas, N. Yoshida. *Less is More: Multiparty Session Types Revisited*. *Proc. ACM Program. Lang.* 3(POPL), 2019.
- [12] K. Honda, V. T. Vasconcelos, M. Kubo. *Language Primitives and Type Discipline for Structured Communication-Based Programming*. *ESOP*, LNCS 1381, pp. 122–138, 1998.
- [13] K. Honda, N. Yoshida, M. Carbone. *Multiparty Asynchronous Session Types*. *J. ACM* 63(1), 2016.
- [14] S. Gay, M. Hole. *Subtyping for Session Types in the Pi Calculus*. *Acta Informatica* 42(2–3), 2005.
- [15] D. Brand, P. Zafriropulo. *On Communicating Finite-State Machines*. *J. ACM* 30(2), 1983.
- [16] E. Li, F. Stutz, T. Wies, D. Zufferey. *Complete Multiparty Session Type Projection with Automata*. *CAV* 2023.
- [17] R. Fikes, N. Nilsson. *STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving*. *Artif. Intell.* 2(3–4), 1971.
- [18] B. Bollig, M. Függer, T. Nowak. *Provable Coordination for LLM Agents via Message Sequence Charts*. *ISoLA*, 2026. arXiv:2604.17612.
- [19] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, L. T. Le. *VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation*. arXiv:2510.05156, 2025.
- [20] V. P. Bhardwaj. *Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents*. arXiv:2602.22302, 2026.
- [21] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, M. Gupta. *SkillSpector: A Pre-Publication Security Control for Agent Skills*. *Agent Skills Workshop at CAIS*, 2026.
- [22] NVIDIA. *NVIDIA-Verified Agent Skills Provide Capability Governance for AI Agents*. NVIDIA Technical Blog, 19 May 2026.



## 333 A Syntax

334 We fix disjoint sets of predicates  $p \in \text{Pred}$  (boolean), numeric variables  $x \in \text{Var}$ , roles  $p, q, r \in \mathcal{R}$ ,  
 335 capability names  $a \in \text{Cap}$ , and message labels  $\ell \in \text{Lab}$ . Following the modern presentation of  
 336 multiparty sessions [1, 2] we proceed bottom-up: state and capabilities (Sections A.1–A.2), then  
 337 processes — the programs skills execute as — (Section A.3), then the global type they are checked  
 338 against directly (Section A.4). In each grammar, “ $::=$ ” lists the possible forms of an expression. In  
 339 each inference rule below, the statement beneath the line follows when every premise above the line  
 340 holds. Rule names identify the corresponding checker obligation; they are not additional assumptions.

### 341 A.1 State logic

$$\begin{aligned} e &::= x \mid n \mid e + e \mid e - e \mid c \cdot e & (c, n \in \mathbb{Z}) \\ \varphi &::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid e \bowtie e & \bowtie \in \{<, \leq, =, \geq, >, \neq\} \end{aligned}$$

342 The state logic  $\mathcal{L}$  is quantifier-free linear integer arithmetic with uninterpreted boolean predicates  
 343 — a decidable fragment for which entailment  $\models$  and satisfiability are discharged by a satisfiability-  
 344 modulo-theories (SMT) solver.

### 345 A.2 Worlds and capabilities

346 A world  $W = \langle B, N \rangle$  assigns truth values to predicates  $B : \text{Pred} \rightarrow \mathbb{B}$  and numeric variables  
 347  $N : \text{Var} \rightarrow \mathbb{Z}$ . A capability context  $\Gamma$  maps each available tool  $a$  to a guarded effect:

$$\begin{aligned} \Gamma(a) &= \langle pre_a, eff_a \rangle, \quad pre_a \in \mathcal{L}, \\ eff_a &= \langle Add_a \subseteq \text{Pred}, Del_a \subseteq \text{Pred}, Asg_a : \text{Var} \rightarrow e, Nd_a : \text{Var} \rightarrow \mathcal{L} \rangle \end{aligned}$$

348  $a \notin \text{dom}(\Gamma) \iff$  the agent does *not* possess tool  $a$  (no hallucinated tools).

349 *Add/Del* are the STRIPS add- and delete-lists; *Asg* gives deterministic numeric updates  $x := e$ ; *Nd*  
 350 gives *nondeterministic* updates  $x := *$  constrained by a formula over the new value (used to model  
 351 post-conditions such as “*books a fare under 500*”).

### 352 A.3 Processes and multiparty sessions

353 Skills *execute* as processes; a multi-agent run is a session of named processes. We use the synchronous  
 354 multiparty session calculus in its modern coinductive presentation [1, 2]: **processes coinductively**  
 355 **defined are possibly infinite regular trees (finitely many distinct subtrees)**, and labels are goal-relevant  
 356 tokens,  $\ell = \alpha(m)$  for concrete messages  $m$  — the tolerance dial  $\alpha$  is a parameter of the label  
 357 alphabet.

$$P ::=^{coind} \mathbf{0} \mid p!\{\ell_i.P_i\}_{i \in I} \mid p?\{\ell_i.P_i\}_{i \in I} \mid a.P \quad \mathbb{M} = p_1[P_1] \parallel \cdots \parallel p_n[P_n]$$

358 with  $I$  finite and non-empty, labels  $\ell_i$  pairwise distinct, and **role** names pairwise distinct. The output  
 359  $p!\{\ell_i.P_i\}$  *internally* chooses a label and sends it to  $p$ ; the input  $p?\{\ell_i.P_i\}$  offers an *external* choice;  
 360  $a.P$  invokes capability  $a$  — the only construct that changes the world. A skill  $S_p$  is a process: the  
 361 behaviour declared for role  $p$ . **why do you need skills? they are only used in C.3 and in a code where**  
 362 **I think is would be better to use processes**

### 363 A.4 Global types

364 The single communication construct of a global type is a *directed* labelled choice — selection by  $p$   
 365 and branching at  $q$  in one construct. (A partnerless “ $p$  selects” admits no coherent view for the roles  
 366 that did not choose; directed choice is the standard MPST construct [13, 9].)

$$G ::=^{coind} p \rightarrow q : \{\ell_i.G_i\}_{i \in I} \mid a@p.G \mid \check{\varphi}.G \mid \text{End}$$

367 subject to  $p \neq q$  and the same regularity and label conditions as processes. This is the *only* type  
 368 in the discipline: Section C.2 types a whole session directly against  $G$  — there is no local-type  
 369 grammar, no projection function, and no separate subtyping relation. Goal markers  $\check{\varphi}$  live *only*  
 370 in  $G$ : achievability is decided on the global type (Rule ACH in C.3), and markers are consumed at  
 371 typing time by T-GOAL without ever burdening a process. Only  $a@p$  changes the world; messages  
 372 carry coordination information only.

## B Operational Semantics

### B.1 World transitions

A capability fires when its guard holds, producing a successor world. Because  $Nd$  is nondeterministic,  $\langle W \rangle a \langle W' \rangle$  may relate one world to many. The relation is implicitly indexed by  $\Gamma$ : the notation  $pre_a, eff_a$  is defined only when  $\Gamma(a) = \langle pre_a, eff_a \rangle$ , so every firing presupposes  $a \in \text{dom}(\Gamma)$ .

$$\frac{\text{WORLD-ACT} \quad W \models pre_a \quad W' \in \text{apply}(eff_a, W)}{\langle W \rangle a \langle W' \rangle}$$

$$\text{apply}(eff_a, \langle B, N \rangle) = \langle B', N' \rangle, \quad B' = (B \cup Add_a) \setminus Del_a$$

$$N'(x) = \begin{cases} \llbracket A sg_a(x) \rrbracket_N & \text{if } x \in \text{dom}(A sg_a) \\ v \text{ with } \langle B, N[x \mapsto v] \rangle \models Nd_a(x) & \text{if } x \in \text{dom}(Nd_a) \\ N(x) & \text{otherwise (frame)} \end{cases}$$

**Mariangiola: where is  $\llbracket A sg_a(x) \rrbracket_N$  defined?**

### B.2 A labelled semantics for sessions and global types

Only capabilities touch the world, so sessions reduce as configurations  $(\mathbb{M}; W)$ ; communication is synchronous [10]. Each transition carries a label  $\Lambda$  recording its *participants*, so that independent moves by disjoint roles can be identified. Two auxiliary maps drive the labelled system: the *participants*  $\text{prt}(\cdot)$  of a global type, a label, or a session, and the *capabilities*  $\text{cap}(\cdot)$  (the set of moves a global type can perform). On global types,

$$\begin{aligned} \text{prt}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p, q\} \cup \bigcup_{i \in I} \text{prt}(G_i) & \text{prt}(\check{\varphi}.G) &= \text{prt}(G) \\ \text{prt}(a @ p.G) &= \{p\} \cup \text{prt}(G) & \text{prt}(\text{End}) &= \emptyset, \end{aligned}$$

on labels  $\text{prt}(p \ell q) = \{p, q\}$ ,  $\text{prt}(a @ p) = \{p\}$ ,  $\text{prt}(\check{\varphi}) = \emptyset$ , and on sessions  $\text{prt}(\mathbb{M}) = \{p \mid \mathbb{M} \equiv p[P] \ \& \ P \neq 0\}$  (the roles with a residual behaviour). The capabilities of a global type are

$$\begin{aligned} \text{cap}(p \rightarrow q : \{\ell_i. G_i\}_{i \in I}) &= \{p \ell_i q \mid i \in I\} \cup \bigcup_{i \in I} \text{cap}(G_i) \\ \text{cap}(a @ p.G) &= \{a @ p\} \cup \text{cap}(G) & \text{cap}(\check{\varphi}.G) &= \{\check{\varphi}\} \cup \text{cap}(G) & \text{cap}(\text{End}) &= \emptyset. \end{aligned}$$

The capability names used by a protocol are

$$\text{caps}(G) = \{a \mid \exists p. a @ p \in \text{cap}(G)\}.$$

**The LTS of sessions is defined by**

$$\begin{array}{c} \text{S-COMM} \\ \hline k \in I \subseteq J \\ \hline (p[q!\{\ell_i. P_i\}_{i \in I}] \parallel q[p?\{\ell_j. Q_j\}_{j \in J}] \parallel \mathbb{M}; W) \xrightarrow{p \ell_k q} (p[P_k] \parallel q[Q_k] \parallel \mathbb{M}; W) \\ \\ \text{S-ACT} \qquad \qquad \qquad \text{S-GOAL} \\ \hline \langle W \rangle a \langle W' \rangle \qquad \qquad \qquad W \models \varphi \\ \hline (p[a.P] \parallel \mathbb{M}; W) \xrightarrow{a @ p} (p[P] \parallel \mathbb{M}; W') \qquad \qquad (\mathbb{M}; W) \xrightarrow{\check{\varphi}} (\mathbb{M}; W) \end{array}$$

**where  $\parallel$  is commutative, associative** and  $p[0] \parallel \mathbb{M} \equiv \mathbb{M}$ . A non-terminated configuration with no successor is *stuck*; the deadlocking planner/worker handoff of Section 1 is the stuck two-role configuration in which both processes begin with an input. A goal marker is *not* carried by any process; instead S-GOAL lets a configuration *observe* that its world already entails one of its declared goals  $\varphi$ , emitting the label  $\check{\varphi}$  without altering  $\mathbb{M}$  or  $W$ . (Each session carries a set of goal formulas, and  $\varphi$  ranges over that set — not over every formula the world happens to satisfy — so that each observation has a matching marker in the protocol.) This observable goal step is matched at the type level by G-GOAL-E below, so that  $\check{\varphi}$  is a genuine label  $\Lambda$  of both transition systems and the operational correspondence of Section D.3 ranges over it uniformly.

The checker relates a session to its protocol through a *labelled* transition system on global types,  $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ , built from *head* rules that consume the leading construct and *interleaving* rules that let a move by participants *disjoint* from the head commute inward:

$$\begin{array}{c}
\text{G-COMM-E} \quad \frac{k \in I}{(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{\mathbf{p} \ell_k \mathbf{q}} (G_k, W)} \quad \text{G-ACT-E} \quad \frac{\langle W \rangle a \langle W' \rangle}{(a @ \mathbf{p}. G, W) \rightsquigarrow_{a @ \mathbf{p}} (G, W')} \\
\\
\text{G-GOAL-E} \quad \frac{W \models \varphi}{(\check{\varphi}. G, W) \rightsquigarrow_{\check{\varphi}} (G, W)} \\
\\
\text{G-COMM-I} \quad \frac{(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W') \quad \Lambda \in \text{cap}(G_i) \quad \forall i \in I \quad \text{prt}(\Lambda) \cap \{\mathbf{p}, \mathbf{q}\} = \emptyset}{(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I}, W) \rightsquigarrow_{\Lambda} (\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G'_i\}_{i \in I}, W')} \\
\\
\text{G-ACT-I} \quad \frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G) \quad \text{prt}(\Lambda) \cap \{\mathbf{p}\} = \emptyset}{(a @ \mathbf{p}. G, W) \rightsquigarrow_{\Lambda} (a @ \mathbf{p}. G', W')} \\
\\
\text{G-GOAL-I} \quad \frac{(G, W) \rightsquigarrow_{\Lambda} (G', W') \quad \Lambda \in \text{cap}(G)}{(\check{\varphi}. G, W) \rightsquigarrow_{\Lambda} (\check{\varphi}. G', W')}
\end{array}$$

396 G-GOAL-E discharges a goal marker when  $W \models \varphi$ , emitting the observable label  $\check{\varphi}$  that matches  
397 S-GOAL. In the head-only reduction used for achievability, an unsatisfied marker is a checkpoint and  
398 blocks the path. In the full labelled correspondence, G-GOAL-I lets a continuation step commute  
399 inward under a still-pending marker. When that step changes the world, the correspondence theorem  
400 uses the goal-stability hypothesis stated in Section D.3; removing that hypothesis is future work. The  
401 extraction (head) rules G-COMM-E/G-ACT-E/G-GOAL-E give the  $\Lambda$ -erased reduction  $(G, W) \rightsquigarrow$   
402  $(G', W')$  that the achievability judgment below uses; the interleaving rules matter only for the  
403 operational correspondence of Section D.3, and their  $\Lambda \in \text{cap}(G)$  premise confines a commuting  
404 move to a capability the residual protocol actually offers. G-COMM-E is legitimate because choice  
405 is directed — the branch is a communication both endpoints observe, and Section C.2 checks that  
406 every third party can follow. A well-formed pack uses its declared goal  $\varphi_{\text{goal}}$  at each goal marker. A  
407 configuration is *goal-satisfying* when control is at such a marker or at the terminus and the world  
408 entails that declared goal:

$$\begin{array}{c}
\text{goal}_{\varphi_{\text{goal}}}(\check{\varphi}_{\varphi_{\text{goal}}}. G, W) \iff W \models \varphi_{\text{goal}}, \quad \text{goal}_{\varphi_{\text{goal}}}(\text{End}, W) \iff W \models \varphi_{\text{goal}}, \\
409 \quad \Gamma; G \models \Diamond_{\text{init}} \varphi_{\text{goal}} \iff \exists W_0 \models \text{init}. \exists (G', W'). (G, W_0) \rightsquigarrow^* (G', W') \wedge \text{goal}_{\varphi_{\text{goal}}}(G', W').
\end{array}$$

410 The diamond is *existential*: a single witnessing run suffices — the formal source of detour-tolerance.

### 411 B.3 Realizability and subtyping, without projection

412 The classical route to ruling out a deadlocking handoff is *projection*: compute each role's local  
413 type  $G|_r$  by structural recursion on  $G$ , taking the *merge* of a choice's branches at every role that  
414 neither sends nor receives, and declaring  $G$  *realizable* exactly when this partial function is defined  
415 everywhere — undefinedness of the merge is the static signature of a role forced to behave differently  
416 in branches it cannot distinguish. Session subtyping  $\leq$  is then a second, separate coinductive relation  
417 on local types, letting a role's actual behaviour offer more external choices and fewer internal ones  
418 than its projected contract demands [14, 10, 11].

419 Both devices exist to answer questions about the *network as a whole* — “can every role tell the  
420 branches apart it needs to?”, “may this implementation stand in for that contract?” — while type-  
421 checking only one role's syntax at a time. Section C.2 answers the same two questions with a single  
422 judgment, *typing the whole configuration directly against G*: a choice node's rule quantifies over  
423 every branch the protocol declares and requires the *same* residual configuration to check against each  
424 one. This is the conservative plain-merge condition, obtained without ever computing  $\sqcap$  or a local  
425 type. The receiver-side subtyping direction (a receiver may accept a superset of the protocol's labels)

is folded into the same rule as a label-set inclusion  $I \subseteq J$ . Tolerance therefore still has the same three independent knobs from Section 2 — the existential  $\diamond$  (Section B.2), subtyping (now inside T-COMM, Section C.2), and the granularity of  $\alpha$  — “flexible, tolerant of small details” is still the profile (coarse  $\alpha$ ,  $\diamond$ , linear  $\mathcal{L}$ ); a later safety layer flips the same machinery to the universal  $\square$  over permission predicates.

## C The Achievability Type System

Achievability now factors into three premises, each separately decidable, combined by the top rule ACH: no hallucinated capability, direct conformance of the declared skills to  $G$ , and reachability of the goal.

### C.1 Capability and goal typing

$$\begin{array}{c} \text{T-EFF} \\ \frac{\Gamma(a) = \langle pre, eff \rangle \quad \varphi \models pre}{\Gamma \vdash \{\varphi\} a \{\text{sp}(\varphi, eff)\}} \end{array} \quad \begin{array}{c} \text{T-MISS} \\ \frac{a \notin \text{dom}(\Gamma)}{\Gamma \vdash a \triangleright \mathbf{X} \text{ (MISSING\_CAPABILITY)}} \end{array}$$

$\text{sp}(\varphi, eff)$  is the strongest postcondition of the STRIPS effect. T-MISS fires on hallucinated planning: the plan names a tool the context does not grant, and achievability is refuted without any search.

### C.2 Direct conformance typing

Skills are typed *directly* against the global protocol and the world, by the coinductive judgment  $G \vdash (\mathbb{M}; W)$  — read “ $G$  types session  $\mathbb{M}$  starting from world  $W$ .” There is no local type, no projection, and no separate subtyping relation: the receiver-side subtyping direction and the unobserved-choice check are structural side conditions of one rule.

$$\begin{array}{c} \text{T-END} \\ \frac{}{\text{End} \vdash (\mathbf{p}[0]; W)} \end{array} \quad \begin{array}{c} \text{T-ACT} \\ \frac{G \vdash (\mathbf{p}[P] \parallel \mathbb{M}; W') \quad \langle W \rangle a \langle W' \rangle \quad \text{prt}(G) \cup \{\mathbf{p}\} = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}\}}{a @ \mathbf{p}. G \vdash (\mathbf{p}[a.P] \parallel \mathbb{M}; W)} \end{array}$$

$$\begin{array}{c} \text{T-GOAL} \\ \frac{G \vdash (\mathbb{M}; W) \quad W \models \varphi \quad \text{prt}(G) = \text{prt}(\mathbb{M})}{\checkmark \varphi. G \vdash (\mathbb{M}; W)} \end{array}$$

$$\begin{array}{c} \text{T-COMM} \\ \frac{G_i \vdash (\mathbf{p}[P_i] \parallel \mathbf{q}[Q_i] \parallel \mathbb{M}; W) \quad \forall i \in I \subseteq J \quad \text{prt}(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I}) = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}, \mathbf{q}\}}{\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}_{i \in I} \vdash (\mathbf{p}[\mathbf{q}!\{\ell_i.P_i\}_{i \in I}] \parallel \mathbf{q}[\mathbf{p}?\{\ell_j.Q_j\}_{j \in J}] \parallel \mathbb{M}; W)} \end{array}$$

T-ACT pairs type and world in lockstep with WORLD-ACT: the unconsumed type  $a @ \mathbf{p}.G$  sits with the *pre*-effect world  $W$  and the residual  $G$  with the *post*-effect world  $W'$  — the same convention as G-ACT-E (Section B.2). In T-COMM the sender  $\mathbf{p}$  offers exactly the protocol’s branch set  $I$ , while the receiver  $\mathbf{q}$  may accept *more* ( $I \subseteq J$ ) — the one subtyping direction the rule keeps (safe interface slack at the receiver). Requiring *every* protocol branch  $\ell_i$  to be checked against the *same* bystanders  $\mathbb{M}$  is the plain-merge form of the unobserved-choice condition, so a role forced to behave differently across branches it cannot distinguish makes no derivation possible — the rule fails closed, and deadlock-freedom falls out of T-COMM itself with no merge to compute (Theorem 4). Each rule additionally carries a *participant-matching* side condition tying the protocol’s participants to the session’s:  $\text{prt}(\mathbf{p} \rightarrow \mathbf{q} : \{\ell_i.G_i\}) = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}, \mathbf{q}\}$  in T-COMM,  $\text{prt}(G) \cup \{\mathbf{p}\} = \text{prt}(\mathbb{M}) \cup \{\mathbf{p}\}$  in T-ACT, and  $\text{prt}(G) = \text{prt}(\mathbb{M})$  in T-GOAL. These make the invariant “the roles the protocol still mentions are exactly the roles still running” hold at every node, ruling out a session that carries a stray participant the protocol has forgotten (or vice versa) — the well-formedness that lets the interleaving cases of Section D.3 match a bystander move on the session to a  $\text{cap}(\cdot)$ -labelled move on the type. T-END closes the discipline at the terminus: the terminated protocol  $\text{End}$  types a single idle role  $\mathbf{p}[0]$  — and, through the congruence  $\mathbf{p}[0] \parallel \mathbb{M} \equiv \mathbb{M}$ , any session all of whose roles have terminated — the base case of the participant invariant, since  $\text{prt}(\text{End}) = \emptyset$  and an idle role contributes nothing to  $\text{prt}(\mathbb{M})$ . T-ACT is derivable only when  $\langle W \rangle a \langle W' \rangle$  holds, which itself presupposes  $a \in \text{dom}(\Gamma)$ : this is where hallucinated capabilities die, with T-MISS as the diagnostic.

### 458 C.3 The achievability judgment

$$\text{ACH} \quad \frac{\text{caps}(G) \subseteq \text{dom}(\Gamma) \quad \exists W_0 \models \text{init}. G \vdash (\mathsf{p}_1[S_{\mathsf{p}_1}] \parallel \dots \parallel \mathsf{p}_n[S_{\mathsf{p}_n}]; W_0) \wedge \Gamma; (G, W_0) \models \Diamond \varphi_{\text{goal}}}{\Gamma \vdash \{S_{\mathsf{p}}\}_{\mathsf{p}} : G \triangleright \Diamond \varphi_{\text{goal}}}$$

459 **Mariangiola: who is p?** Read top-to-bottom: no hallucinated tools; some plausible initial world exists  
 460 from which the declared skills directly conform to the protocol (deadlock-free, every capability call  
 461 guarded) *and* that same world reaches the goal. The reachability half is unchanged from Section B.2  
 462 and decided on  $\Gamma, G$  alone, before any  $S_{\mathsf{p}}$  is known — exactly what lets the checker run on the  
 463 LLM-compacted pack the moment it exists; conformance is the additional, separately decidable  
 464 check once the skills are declared, and needs no transport lemma: the receiver-side subtyping slack is  
 465 already inside T-COMM.

### 466 C.4 Reachability rules

The diamond is derived by the following inductive system, realized by the checker as a finite forward search over  $\rightsquigarrow$ ; unlike T-COMM/T-ACT/ T-GOAL, it never mentions a session  $\mathbb{M}$ , which is what makes it decidable on the pack alone.

$$\begin{array}{c} \text{REACH-GOAL} \\ \frac{W \models \varphi_{\text{goal}} \quad G = \text{End} \vee \exists G'. G = \check{\varphi}_{\text{goal}}. G'}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} \\[10pt] \begin{array}{cc} \text{REACH-STEP} & \text{REACH-OR} \\ \frac{(G, W) \rightsquigarrow (G', W') \quad \Gamma; (G', W') \models \Diamond \varphi_{\text{goal}}}{\Gamma; (G, W) \models \Diamond \varphi_{\text{goal}}} & \frac{\exists k \in I. \Gamma; (G_k, W) \models \Diamond \varphi_{\text{goal}}}{\Gamma; (\mathsf{p} \rightarrow \mathsf{q}; \{\ell_i.G_i\}_{i \in I}, W) \models \Diamond \varphi_{\text{goal}}} \end{array} \end{array}$$

## 467 D Metatheory

468 We separate what is *mechanized in Coq* (Theorems 1–3; Theorems 4–5 for the single-label com-  
 469 munication fragment with full interleaving; the head-move action case; and the concrete instances,  
 470 all axiom-free) from what is *proved on paper* (the action-interleaving cases, decidability, and the  
 471 undecidability boundary).

### 472 D.1 Soundness of the abstraction

473 Let  $\mathcal{C}$  be the concrete configuration space of protocol/world pairs,  $\mathcal{A}$  the abstract configuration space  
 474 the checker explores, and  $\text{abs} : \mathcal{C} \rightarrow \mathcal{A}$  the abstraction. We require:

$$(\text{step-sim}) \quad C \rightsquigarrow C' \Rightarrow \text{abs}(C) \rightsquigarrow^* \text{abs}(C'), \quad (\text{goal-sim}) \quad \text{goal}_{\varphi_{\text{goal}}}(C) \Rightarrow \hat{\text{goal}}(\text{abs}(C)).$$

475 **Lemma 1** (Transport). *If  $C_0 \rightsquigarrow^* C$  then  $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$ .*

476 *Proof.* By induction on the reflexive-transitive closure. The empty run is reflexivity. For  $C_0 \rightsquigarrow^* C'$   
 477  $C' \rightsquigarrow C$ , the induction hypothesis gives  $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C')$  and STEP-SIM gives  $\text{abs}(C') \rightsquigarrow^* \text{abs}(C)$ ;  
 478 compose them. Mechanized as `reach_abs`, parameterized by STEP-SIM.  $\square$

479 **Lemma 2** (Reachability monotonicity). *If every step of  $\rightsquigarrow_1$  is a step of  $\rightsquigarrow_2$  (that is,  $x \rightsquigarrow_1 y$  implies  
 480  $x \rightsquigarrow_2 y$ ), then  $s \rightsquigarrow_1^* w$  implies  $s \rightsquigarrow_2^* w$ .*

481 *Proof.* By induction on the closure  $s \rightsquigarrow_1^* w$ . The empty run transports by reflexivity. For  $s \rightsquigarrow_1^* u \rightsquigarrow_1 w$ ,  
 482 the induction hypothesis gives  $s \rightsquigarrow_2^* u$ ; the hypothesis turns the last step  $u \rightsquigarrow_1 w$  into  $u \rightsquigarrow_2 w$ ;  
 483 appending it gives  $s \rightsquigarrow_2^* w$ . Mechanized as `reach_mono`.  $\square$

## 484 D.2 Refutation soundness, tolerance, monotonicity

485 **Theorem 1** (Refutation soundness). *If the abstract system has no reachable goal state from*  
 486  *$\text{abs}(G, W_0)$ , then no declared run of that pack configuration reaches  $\varphi_{\text{goal}}$ . Hence an impossi-*  
 487 *ble verdict is never wrong relative to the pack, provided STEP-SIM and GOAL-SIM hold.*

488 *Proof.* We prove the contrapositive: if *some* declared run reaches the goal, then the abstract  
 489 system reaches an abstract goal state. Suppose  $C_0 \rightsquigarrow^* C$  with  $\text{goal}_{\varphi_{\text{goal}}}(C)$ . By Lemma 1,  
 490  $\text{abs}(C_0) \rightsquigarrow^* \text{abs}(C)$ , and by GOAL-SIM,  $\hat{\text{goal}}(\text{abs}(C))$ . Thus  $\text{abs}(C)$  is an abstract goal state reach-  
 491 able from  $\text{abs}(C_0)$ , contradicting the hypothesis. Hence no declared run reaches  $\varphi_{\text{goal}}$ : a verdict  
 492 of *impossible* — read off exactly the premise, that the abstract search exhausts without hitting an  
 493 abstract goal — is never wrong. Mechanized, axiom-free as a theorem schema parameterized by the  
 494 two simulation hypotheses (the existential witness is  $\text{abs}(C)$ ):  $\square$

```
495   Theorem refutation_sound : forall s0,
496     (~ exists a, reach astep (abs s0) a /\ agoal a) ->
497     (~ exists w, reach cstep s0 w /\ cgoal w).
498     (* Print Assumptions refutation_sound. => Closed under the global
499     context *)
```

500 **Theorem 2** (Tolerance soundness). *If a coarser over-approximation (more edges) cannot reach the*  
 501 *goal, neither can any finer system. Hence coarsening  $\alpha$  to tolerate more detail never produces a false*  
 502 *refutation.*

503 *Proof.* Let  $\rightsquigarrow_{\text{fine}}$  and  $\rightsquigarrow_{\text{coarse}}$  be the two abstract step relations, with the coarsening an *over-*  
 504 *approximation*: every fine step is also a coarse step,  $x \rightsquigarrow_{\text{fine}} y \Rightarrow x \rightsquigarrow_{\text{coarse}} y$ . Suppose, for  
 505 contradiction, that the finer system reaches the goal:  $s_0 \rightsquigarrow_{\text{fine}}^* a$  with  $\hat{\text{goal}}(a)$ . By Lemma 2 (with  
 506  $\rightsquigarrow_1 = \rightsquigarrow_{\text{fine}}$ ,  $\rightsquigarrow_2 = \rightsquigarrow_{\text{coarse}}$ ),  $s_0 \rightsquigarrow_{\text{coarse}}^* a$ ; and  $\hat{\text{goal}}(a)$  still holds. So the coarser system reaches  
 507 a goal state, contradicting the hypothesis. Hence no coarser-refuted goal is reachable in any finer  
 508 system: coarsening  $\alpha$  only *adds* abstract edges, so it can never turn an unreachable goal into a  
 509 reachable one — refutation is preserved. Mechanized as `tolerance_sound`, axiom-free.  $\square$

510 **Theorem 3** (Capability monotonicity). *If  $\Gamma \subseteq \Gamma'$  and  $\Gamma$  reaches  $\varphi_{\text{goal}}$ , then  $\Gamma'$  reaches  $\varphi_{\text{goal}}$ .*  
 511 *Granting tools never makes an achievable goal impossible.*

512 *Proof.* Enlarging the capability context only *adds* guarded effects: since  $\Gamma \subseteq \Gamma'$ , every capability  
 513 firing available under  $\Gamma$  is available under  $\Gamma'$  with the same guard and effect (WORLD-ACT in B.1  
 514 quantifies over  $a \in \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$ ), so every step of the  $\Gamma$ -transition relation is a step of  
 515 the  $\Gamma'$ -relation. Suppose  $\Gamma$  reaches the goal configuration  $C$  from  $C_0$ . By Lemma 2, the same  
 516 configuration is reachable under  $\Gamma'$ , and  $\text{goal}_{\varphi_{\text{goal}}}(C)$  is unchanged because the goal predicate does  
 517 not depend on  $\Gamma$ . Hence  $\Gamma'$  reaches  $\varphi_{\text{goal}}$  via the same witnessing run. The other premises of ACH  
 518 are monotone in the same direction ( $\text{caps}(G) \subseteq \text{dom}(\Gamma) \subseteq \text{dom}(\Gamma')$  still holds, and realizability  
 519 and conformance do not mention  $\Gamma$  beyond  $a \in \text{dom}(\Gamma)$ ), so the whole judgment is preserved. The  
 520 transport half is mechanized as `cap_monotone`, axiom-free, stated over an inclusion of step relations;  
 521 the reduction of  $\Gamma \subseteq \Gamma'$  to that inclusion is the WORLD-ACT argument above, on paper.  $\square$

522 **Concrete instance.** The flight skill of Section 2, variant A, is mechanized as `FlightInstance`. We  
 523 prove `flight_refuted` (no run reaches `booked`  $\wedge$  `confirmation_sent`, since no step touches `conf`)  
 524 and `booking_reachable` (a booking is still reachable). Both are axiom-free, witnessing non-vacuity  
 525 of Theorem 1.

## 526 D.3 Operational correspondence: subject reduction and session fidelity

527 The direct judgment  $G \vdash (\mathbb{M}; W)$  is not merely preserved by reduction: the session and its protocol  
 528 step *in lockstep* over the labelled transition systems of Section B.2. This is the standard soundness  
 529 backbone of a session calculus [13, 11], obtained here *directly*, with no projection and no merge. For  
 530 world-changing interleavings, the statements below assume two explicit frame conditions: effects of  
 531 participant-disjoint actions commute, and an action moved under a pending marker preserves that  
 532 marker's formula. Communication-only reductions satisfy both conditions vacuously.



533 **Lemma 3** (Residual capability). *If  $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ , then  $\Lambda \in \text{cap}(G)$ .*

534 **Lemma 4** (Participant agreement). *If  $G \vdash (\mathbb{M}; W)$ , then  $\text{prt}(G) = \text{prt}(\mathbb{M})$ .*

535 Both follow by induction on the transition or typing derivation, respectively; they are the capability-  
536 membership and participant invariants used in the interleaving cases below.

537 **Theorem 4** (Subject reduction). *If  $G \vdash (\mathbb{M}; W)$  and  $(\mathbb{M}, W) \xrightarrow{\Lambda} (\mathbb{M}', W')$ , then  $(G, W) \rightsquigarrow_{\Lambda}$   
538  $(G', W')$  for some  $G'$  with  $G' \vdash (\mathbb{M}'; W')$ .*

539 **Theorem 5** (Session fidelity). *If  $G \vdash (\mathbb{M}; W)$  and  $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ , then  $(\mathbb{M}, W) \xrightarrow{\Lambda}$   
540  $(\mathbb{M}', W')$  for some  $\mathbb{M}'$  **and**  $W'$  with  $G' \vdash (\mathbb{M}'; W')$ .*

541 We prove both by induction on the typing derivation  $G \vdash (\mathbb{M}; W)$ , with a case analysis on the  
542 transition; the one auxiliary fact used throughout is that typing respects pointwise-equal sessions  
543 (`ctypes_ext`), which lets the interleaving cases reassociate independent updates.

544 *Proof of Theorem 4 (subject reduction).* By induction on  $G \vdash (\mathbb{M}; W)$ .

545 *Case T-END* ( $G = \text{End}$ , every role idle). Then  $\mathbb{M}$  has no output or pending action, so the only  
546 available transition is the goal observation S-GOAL at  $\Lambda = \checkmark\varphi_{\text{goal}}$  when  $W \models \varphi_{\text{goal}}$ , which leaves  
547  $(\mathbb{M}, W)$  fixed; there is no residual type to preserve and the claim holds (otherwise no  $\Lambda$ -transition  
548 exists and it holds vacuously).

549 *Case T-GOAL* ( $G = \checkmark\varphi.G_0$ , with  $W \models \varphi$  and  $G_0 \vdash (\mathbb{M}; W)$ ,  $\text{prt}(G_0) = \text{prt}(\mathbb{M})$ ). The transition  
550  $(\mathbb{M}, W) \rightarrow_{\Lambda} (\mathbb{M}', W')$  splits on its label.

551 • *Goal observation* ( $\Lambda = \checkmark\varphi$ , by S-GOAL, so  $\mathbb{M}' = \mathbb{M}$  and  $W' = W$ ). Since  $W \models \varphi$ , rule  
552 G-GOAL-E discharges the marker,  $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\checkmark\varphi} (G_0, W)$ , and  $G_0 \vdash (\mathbb{M}; W)$  is the  
553 premise: take  $G' = G_0$ .

554 • *Interleaving* ( $\Lambda \neq \checkmark\varphi$ ). A goal marker is not a session construct, so this is already a  
555 transition of  $G_0$ 's session; the induction hypothesis gives  $(G_0, W) \rightsquigarrow_{\Lambda} (G', W')$  with  
556  $\Lambda \in \text{cap}(G_0)$  and  $G' \vdash (\mathbb{M}'; W')$ . When the move is world-preserving (a communica-  
557 tion,  $W' = W$ ), G-GOAL-I carries the still-pending marker along,  $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda}$   
558  $(\checkmark\varphi.G', W)$ , and  $\checkmark\varphi.G' \vdash (\mathbb{M}'; W)$  by T-GOAL (the side condition  $\text{prt}(G') = \text{prt}(\mathbb{M}')$  is  
559 preserved because  $\Lambda$  acts on the same participants on both sides). When the move is *world-*  
560 *changing* (a capability firing,  $W' \neq W$ ), G-GOAL-I still commutes it under the marker,  
561  $(\checkmark\varphi.G_0, W) \rightsquigarrow_{\Lambda} (\checkmark\varphi.G', W')$ ; re-typing the residual by T-GOAL then additionally asks  
562  $W' \models \varphi$ . This follows from the goal-stability hypothesis on actions commuting under pend-  
563 ing markers. The global type sequences the marker before  $G_0$ 's actions whereas the session  
564 may fire the action first; the interleaving rule lets the type catch up under that hypothesis.  
565 Removing the hypothesis, rather than this conditional case, is the open correspondence  
566 problem recorded in Section 7.

567 *Case T-COMM*, with head  $p \rightarrow q$  and branches  $G_i$ . Here  $\mathbb{M}$  is  $p[q! \dots] \parallel q[p? \dots] \parallel \mathbb{M}_0$ , and each  
568 branch satisfies  $G_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0; W)$ . If the transition is a goal observation  $\Lambda = \checkmark\varphi$   
569 (S-GOAL, leaving  $(\mathbb{M}, W)$  fixed), it is matched by G-COMM-I:  $\text{prt}(\checkmark\varphi) = \emptyset$  makes the disjointness  
570 premise trivial, and each branch realises the same observation by the induction hypothesis, using  
571  $\checkmark\varphi \in \text{cap}(G_i)$  — the observation concerns a marker the protocol actually pends (see the note below).  
572 Otherwise the transition is a communication  $\Lambda = r \ell t$ . Because  $p$  offers only outputs to  $q$  and  $q$  only  
573 inputs from  $p$ , the sender/receiver shapes force a dichotomy.

574 • *Head move* ( $r = p$ ). Then  $t = q$  and  $\ell = \ell_k \in I$ , and  $\mathbb{M}' = p[P_k] \parallel q[Q_k] \parallel \mathbb{M}_0$ . Rule  
575 G-COMM-E gives  $(G, W) \rightsquigarrow_{p\ell_k q} (G_k, W)$ , and  $G_k \vdash (\mathbb{M}'; W)$  is exactly the  $k$ -th premise.

576 • *Interleaving move* ( $r \neq p$ ). The shapes then force  $r, t \notin \{p, q\}$ :  $r$  is an output so  $r \neq q$ ,  
577 and  $t$  is an input so  $t \neq p$ , while  $t = q$  would force  $r = p$ . Since  $r, t$  lie in the bystanders  
578  $\mathbb{M}_0$ , the *same* communication is enabled in each branch's configuration  $p[P_i] \parallel q[Q_i] \parallel \mathbb{M}_0$   
579 (the head updates do not touch  $r, t$ ). The induction hypothesis at each  $i$  yields  $G'_i$  with  
580  $(G_i, W) \rightsquigarrow_{\Lambda} (G'_i, W)$  and  $G'_i \vdash (p[P_i] \parallel q[Q_i] \parallel \mathbb{M}'_0; W)$ , where  $\mathbb{M}'_0$  is  $\mathbb{M}_0$  after the  
581 bystander step. As  $\text{prt}(\Lambda) = \{r, t\}$  is disjoint from  $\{p, q\}$ , rule G-COMM-I assembles

582  $(G, W) \rightsquigarrow_{\Lambda} (p \rightarrow q; \{\ell_i.G'_i\}, W)$ , and re-applying T-COMM (the sender/receiver at  $p, q$   
 583 are untouched, and each branch is retyped by the induction hypothesis, reassociating the  
 584 independent updates via `ctypes_ext`) gives the residual typing.

585 Communications leave the world fixed, so no independence hypothesis is needed. For a world-  
 586 changing action at the head (T-ACT), G-ACT-E matches and the residual  $G$  is typed at the *same*  
 587 post-effect world the rule reaches — this is exactly where the lockstep world pairing of the corrected  
 588 T-ACT is used; a goal observation at an action-typed configuration is matched by G-ACT-I exactly  
 589 as in the T-COMM case, since  $\text{prt}(\check{\varphi}) \cap \{p\} = \emptyset$  trivially. Interleaving *two* actions requires their  
 590 effects to commute ( $\text{eff}_a \circ \text{eff}_b = \text{eff}_b \circ \text{eff}_a$  for disjoint  $a, b$ ), a frame-independence side condition  
 591 that participant-disjointness alone does not supply; under it the T-ACT interleaving case goes through  
 592 identically.  $\square$

593 *Proof of Theorem 5 (session fidelity).* By induction on  $G \vdash (\mathbb{M}; W)$ , inverting the global step  
 594  $(G, W) \rightsquigarrow_{\Lambda} (G', W')$ . T-END admits only the goal observation G-GOAL-E at  $\check{\varphi}_{goal}$ , matched  
 595 on the session by S-GOAL. For T-GOAL, the step is either G-GOAL-E — matched by S-GOAL  
 596 on the session, the marker discharged — or G-GOAL-I, whose premise  $(G_0, W) \rightsquigarrow_{\Lambda} (G'_0, W')$   
 597 the induction hypothesis realises as an underlying session step that carries the marker along. For  
 598 T-COMM, the global step is either G-COMM-E — and the prescribed head communication  $p \ell_k q$  is  
 599 enabled by S-COMM, landing in the  $k$ -th premise — or G-COMM-I with  $\text{prt}(\Lambda) \cap \{p, q\} = \emptyset$ ; then  
 600 the induction hypothesis on any branch produces a bystander communication, whose endpoints lie  
 601 outside  $\{p, q\}$ , so S-COMM fires it on  $\mathbb{M}$  and T-COMM retypes the result. The action cases mirror  
 602 those of Theorem 4.  $\square$

603 **Mechanization.** Both theorems are mechanized axiom-free for the single-label communication  
 604 fragment — the fragment in which bystander interleaving is unconditional — in `DirectTypingSR.v`,  
 605 over labelled session and global transition systems, with no projection, merge, or local type:

```
606 Theorem subject_reduction : forall W G s s' L,
607   ctypes W G s -> lstep L s s' ->
608   exists G', gstep W L G G' /\ ctypes W G' s'.
609 Theorem session_fidelity : forall W G G' s L,
610   ctypes W G s -> gstep W L G G' ->
611   exists s', lstep L s s' /\ ctypes W G' s'.
612 (* Print Assumptions => Closed under the global context (both). *)
```

613 The head-move case for world-changing *actions* is additionally mechanized in `DirectTyping.v`  
 614 (`type_directed_safety`, also `progress`). Observable goal labels appear in neither Coq develop-  
 615 ment: the mechanized label alphabet is communication-only, and its goal rule fuses marker discharge  
 616 with a continuation step. Participant-matching side conditions, labelled action interleaving, and  
 617 multi-branch choice are likewise absent. Completing a faithful mechanization of the paper rules  
 618 remains work in Section 7.

619 **Concrete instance.** `HandoffInstance` mechanizes the planner/worker handoff of Section 1 on both  
 620 sides. The *good* pairing — planner sends, worker delivers, goal checked — is proved `ctypes`-typed  
 621 (`good_typed`) and its run is proved to reach a world satisfying the goal (`good_runs_to_goal`). The  
 622 *bad* pairing — both roles start with an input, exactly the deadlock of Section 1 — is proved `Stuck`  
 623 (`bad_stuck`), and the contrapositive of `progress` then gives, for free, that *no non-trivial global type*  
 624 *can ever type it* (`bad_untypable`): the rule rejects the classical deadlocking handoff without any  
 625 projection ever being computed.

## 626 D.4 Incompleteness, by design

627 **Proposition 1** (Incompleteness).  $\Gamma; G \models \Diamond_{init} \varphi_{goal}$  may hold while no concrete runtime execution  
 628 reaches  $\varphi_{goal}$ : the witnessing abstract path can be spurious when  $\alpha$  collapses a distinction that  
 629 mattered. The system is sound for  $\times$  and incomplete for  $\checkmark$ .

630 This is the price of decidable, tolerant over-approximation; tightening  $\alpha$  trades tolerance for precision.  
 631 The residue is confined to two edges, motivating the layered architecture:

- **Payload faithfulness** (bottom edge): a tool’s declared post-condition (“filters to fares < 500”) may be false — a runtime obligation for a deterministic monitor (Layer C), not a static one.
- **Intent fidelity** (top edge): the formalized goal may not capture what the user meant — irreducibly semantic, surfaced at the single compaction checkpoint for human review.

## D.5 Decidability and the autonomy boundary

**Theorem 6** (Decidability). *In the fragment with finitely many roles (static topology), regular global types represented by finite tail-recursive control graphs, decidable state logic  $\mathcal{L}$ , and  $\mathcal{L}$ -definable effects,  $\Gamma; G \models \Diamond_{init} \varphi_{goal}$  is decidable.*

*Proof.* The pack-level achievability judgment  $\Gamma; G \models \Diamond_{init} \varphi_{goal}$  (Section B.2) is an existential reachability query over configurations  $(G', W)$ ; we exhibit a terminating decision procedure and argue its completeness.

*The search space is finite.* A configuration has two components. (i) *Control*, the residual global type  $G'$  reachable from  $G$  by  $\rightsquigarrow$ . Since  $G$  is a regular tree (tail-recursion: finitely many distinct subtrees, Section A.3) and each  $\rightsquigarrow$  step either descends into a subtree (G-COMM-E, G-ACT-E) or erases a marker (G-GOAL-E), the set  $\{G' : (G, \_) \rightsquigarrow^* (G', \_)\}$  is a subset of the finitely many subtrees of  $G$ , hence finite. (ii) *Symbolic world*. The boolean part is a valuation  $B : \text{Pred} \rightarrow \mathbb{B}$  over the finitely many predicates named in the pack, so there are  $2^{|\text{Pred}|}$  boolean states. The numeric part is summarized by an accumulated constraint  $\psi \in \mathcal{L}$  (a conjunction of the guards taken and the  $Nd$ -postconditions assumed); on a control-graph back edge the reference procedure applies the widening  $\psi \mapsto \top$ , forgetting accumulated numeric constraints. This one-element numeric summary is an over-approximation and, together with the finite boolean valuations, leaves only finitely many symbolic worlds. The product with finite control is therefore finite.

*Each edge is computable.* Firing a capability requires checking  $W \models pre_a$  and forming  $sp$ ; testing goal-satisfaction requires  $W \models \varphi$ . Both are entailment/satisfiability queries in quantifier-free linear integer arithmetic with uninterpreted predicates — a decidable theory, discharged by an SMT solver. Branch selection (REACH-OR) is a finite disjunction. Existence of an initial world is decided by the same SMT query over  $init$ ; each satisfying symbolic initial valuation seeds the finite search.

*Termination and correctness.* Forward search (breadth-first over  $\rightsquigarrow$ ) maintains a visited set of (control, symbolic-world) pairs; because that set is finite it adds each pair at most once, so the search halts. It answers ACHIEVABLE iff it enumerates a goal-satisfying configuration, and IMPOSSIBLE otherwise. Soundness of the IMPOSSIBLE verdict is Theorem 1; and widening only *enlarges* the reachable set (it weakens constraints, adding edges), so by Theorem 2 it never introduces a false refutation. Hence the procedure is a decision procedure for  $\Gamma; G \models \Diamond_{init} \varphi_{goal}$ .  $\square$

**Proposition 2** (Undecidability in a dynamic-topology extension). *Consider the extension  $G^+$  of the grammar with spawn  $r.G$ , which creates fresh participants at run time. With an unbounded role population and dynamic channels, achievability in  $G^+$  is undecidable.*

*Proof sketch.* By reduction from the control-state reachability problem for communicating finite-state machines (CFSMs), which is undecidable [15]. Fix a CFSM instance: finite-control machines  $M_1, M_2$  exchanging messages over unbounded FIFO channels, and a target control state  $q_f$  of  $M_1$ ; deciding whether a run reaches  $q_f$  is undecidable.

We compute, from this instance, a skill whose goal is achievable iff the CFSM reaches  $q_f$ . Each machine  $M_i$  becomes a role whose process mirrors its control graph: a transition that sends message  $m$  spawns, at run time, a fresh *courier* participant carrying  $m$  (this is exactly the dynamic-spawning capability the theorem hypothesizes), and a transition that receives  $m$  synchronizes with the oldest outstanding courier for that channel, so the unbounded family of couriers realizes the unbounded FIFO. The required order can be made explicit by assigning sequence numbers through the numeric world state and guarding receives on the next sequence number; the *Asg/Nd* machinery of Section A.2 supplies those updates. A boolean predicate  $at_{q_f}$  is established by the single capability guarding  $M_1$ ’s entry into  $q_f$ , and we set  $\varphi_{goal} = at_{q_f}$ . By construction a run of the skill reaches a state satisfying  $\varphi_{goal}$  iff the CFSM has a run reaching  $q_f$ : the courier discipline then puts the skill’s reachable configurations in correspondence with the CFSM’s channel contents and control states.

684 The construction is a computable map from CFSM instances to packs, so a decision procedure for  
 685 achievability would decide CFSM control-state reachability — impossible. Hence achievability is  
 686 undecidable once participants may be spawned dynamically. (The finiteness argument of Theorem 6  
 687 breaks at exactly the point this reduction exploits: an unbounded number of couriers makes the set of  
 688 reachable controls infinite.)  $\square$

689 The two results isolate one concrete boundary: static finite topology admits a total decision procedure,  
 690 while unbounded dynamic spawning does not. This is a boundary on topology, not a claim that  
 691 all forms of agent autonomy are undecidable. The implementation returns UNKNOWN when a  
 692 pack violates the static-topology side condition of Theorem 6, unless an independent capability or  
 693 establisher refutation has already proved it impossible.

## 694 D.6 The partition of obligations

695 **Corollary 1.** *The architecture separates “will the goal be achieved” into four obligations on disjoint*  
 696 *substrates:*

|     | Concern                           | Where                      | Mechanism                                       |
|-----|-----------------------------------|----------------------------|-------------------------------------------------|
| 697 | goal achievability                | static, decidable          | Coq specification; deterministic search; no LLM |
|     | per-step safety / least privilege | Layer B (future)           | same engine, $\square$ + permission tiers       |
|     | payload faithfulness              | Layer C (future), run-time | deterministic monitors with blame               |
|     | intent fidelity                   | top edge, synthesis        | one human checkpoint                            |

## 698 D.7 Verifiable feedback beyond a scalar reward

699 The verdict is a heterogeneous verification signal rather than a scalar score. An IMPOSSIBLE result  
 700 identifies which premise failed and carries a blocking frontier; ACHIEVABLE carries a witness  
 701 path and explicitly retains the two deferred obligations; UNKNOWN is an abstention outside the  
 702 decidable fragment, not a refutation. These structured outcomes can serve as cheap negative labels  
 703 for automated skill or prompt search without an LLM judge, while the intent-fidelity checkpoint  
 704 keeps the irreducible semantic decision with a human. We treat such search as an application of the  
 705 verifier, not as a contribution evaluated here.

## 706 E Token economics of pre-execution refutation

707 The practical case for deciding achievability *before* execution is an economic one, so we state it in  
 708 tokens rather than adjectives. Two asymmetries carry it. First, no model sits in the trusted path: the  
 709 schema gate, projection, conformance and Z3 reachability spend **zero tokens**, and the deterministic  
 710 front-end spends zero as well. Only the optional LLM compaction spends tokens, and it is paid *once*  
 711 *per skill version*. Second, an agent turn re-sends its whole context: writing  $S$  for the harness prompt,  
 712  $K$  for the loaded skill,  $g$  for the mean tokens appended per turn and  $o$  for output per turn, a run of  $T$   
 713 turns reads  $T(S+K) + gT(T-1)/2$  input and  $To$  output tokens. Compaction is linear in the skill  
 714 and paid once; a doomed run is *quadratic* in its turns and recurs on every invocation.

715 Runtime waste is necessarily a model — it prices a run that, if the refutation is correct, never  
 716 happens — so we publish the model rather than a single number. Each refutation reason carries  
 717 a (low, typical, high) band for how long an agent flails before that failure stops it, and how many  
 718 agents are billed. With conservative defaults ( $S=2500$ ,  $g=700$ ,  $o=250$ ) and a median real consumer  
 719 skill ( $K \approx 1.7K$  tokens), compaction costs  $\sim 2.8K$  tokens against the following avoided waste per  
 720 prevented invocation:

|     | Verdict reason     | turns (lo–typ–hi) | waste/run | leverage |
|-----|--------------------|-------------------|-----------|----------|
| 721 | MISSING_CAPABILITY | 3–8–14, 1 agent   | 50 240    | 18×      |
|     | GOAL_UNSAT         | 8–18–30, 1 agent  | 176 040   | 63×      |
|     | NON_CONFORMANT     | 6–15–30, 2 agents | 261 900   | 94×      |
|     | NON_PROJECTABLE    | 6–15–30, 2 agents | 261 900   | 94×      |
|     | BLOCKED_GUARD      | 12–25–50, 1 agent | 305 750   | 110×     |

The ordering is the robust part, and it is the expected one. `MISSING_CAPABILITY` is cheapest at run time: the agent calls a tool that is not there and learns immediately, and most of the cost is the improvisation that follows. `BLOCKED_GUARD` is dearest, because a precondition no run can satisfy is precisely the case an agent cannot learn from — it retries to the harness turn cap. The two multi-party reasons bill two contexts. `GOAL_UNSAT` additionally carries a cost the token bill does not show: a run whose goal conjunct has no establisher can terminate *believing* it succeeded. `DYNAMIC_TOPOLOGY` claims no savings at all, because an abstention prevents nothing.

Over the 15-spec corpus the seven structural refutations cost 12.2K tokens of compaction and avoid  $\sim 1.07\text{M}$  tokens per round of invocations (band 0.28M–3.15M):  $\sim 87\times$  leverage, or nothing at all on the deterministic path. Break-even therefore falls *inside* the first prevented run in every mode — between 0.9% and 5.5% of it. The honest denominator for a healthy skill, where the check buys nothing and still costs something, is 2.9% of one successful run over the corpus and 4.0% for a median real skill; with the deterministic front-end, zero. Prompt caching changes the price of the wasted tokens, not their number. Halving every waste default leaves leverage at  $9\times$ – $55\times$  and the ordering unchanged. The model, its defaults and their justification are in the artifact (`skillc.tokens`, `skillc.cost`); the Coq development costs no tokens and is amortized over every skill ever checked.

## NeurIPS Paper Checklist

### 1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 state that refutation is sound relative to the declared pack, distinguish `UNKNOWN` from refutation, and identify the mechanized and paper-only proof fragments.

### 2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The Limitations and Future Work section discusses relative soundness, static topology, abstraction coarseness, mechanization gaps, and the proof-of-concept corpus.

### 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: Appendix D states the simulation, stability, commutativity, finiteness, and topology assumptions with proofs or proof sketches; the artifact checks the explicitly identified Coq fragments.

### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Sections 4–5 describe the deterministic checker, verdict semantics, corpus construction, binary accounting, and separate treatment of abstentions.

### 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: The anonymized supplemental artifact contains the checker, synthetic corpora, proof developments, pinned continuous-integration workflow, and reproduction commands.

|     |                                                                                                                              |
|-----|------------------------------------------------------------------------------------------------------------------------------|
| 772 | <b>6. Experimental setting/details</b>                                                                                       |
| 773 | Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-                              |
| 774 | rameters, how they were chosen, type of optimizer) necessary to understand the results?                                      |
| 775 | Answer: [N/A]                                                                                                                |
| 776 | Justification: The evaluation is a deterministic execution over declared packs; it performs no                               |
| 777 | training, optimization, sampling, or data splitting.                                                                         |
| 778 | <b>7. Experiment statistical significance</b>                                                                                |
| 779 | Question: Does the paper report error bars suitably and correctly defined or other appropriate                               |
| 780 | information about the statistical significance of the experiments?                                                           |
| 781 | Answer: [N/A]                                                                                                                |
| 782 | Justification: Each corpus case and checker result is deterministic, so repeated trials have no                              |
| 783 | sampling variance; the paper reports exact counts rather than inferential statistics.                                        |
| 784 | <b>8. Experiments compute resources</b>                                                                                      |
| 785 | Question: For each experiment, does the paper provide sufficient information on the com-                                     |
| 786 | puter resources (type of compute workers, memory, time of execution) needed to reproduce                                     |
| 787 | the experiments?                                                                                                             |
| 788 | Answer: [Yes]                                                                                                                |
| 789 | Justification: Section 5 states that evaluation is CPU-only, requires no model inference or                                  |
| 790 | training, and gives the corpus size and solver timeout governing the run.                                                    |
| 791 | <b>9. Code of ethics</b>                                                                                                     |
| 792 | Question: Does the research conducted in the paper conform, in every respect, with the                                       |
| 793 | NeurIPS Code of Ethics <a href="https://neurips.cc/public/EthicsGuidelines?">https://neurips.cc/public/EthicsGuidelines?</a> |
| 794 | Answer: [Yes]                                                                                                                |
| 795 | Justification: The work uses synthetic packs, no personal data or human subjects, and makes                                  |
| 796 | bounded claims with explicit trust assumptions and limitations.                                                              |
| 797 | <b>10. Broader impacts</b>                                                                                                   |
| 798 | Question: Does the paper discuss both potential positive societal impacts and negative                                       |
| 799 | societal impacts of the work performed?                                                                                      |
| 800 | Answer: [Yes]                                                                                                                |
| 801 | Justification: The Broader impact paragraph discusses reduced wasted execution as well as                                    |
| 802 | false confidence from inaccurate packs and the human/runtime mitigations required.                                           |
| 803 | <b>11. Safeguards</b>                                                                                                        |
| 804 | Question: Does the paper describe safeguards that have been put in place for responsible                                     |
| 805 | release of data or models that have a high risk for misuse (e.g., pre-trained language models,                               |
| 806 | image generators, or scraped datasets)?                                                                                      |
| 807 | Answer: [N/A]                                                                                                                |
| 808 | Justification: The artifact releases no trained model, scraped dataset, or other high-risk asset;                            |
| 809 | it contains a deterministic checker and synthetic examples.                                                                  |
| 810 | <b>12. Licenses for existing assets</b>                                                                                      |
| 811 | Question: Are the creators or original owners of assets (e.g., code, data, models), used in                                  |
| 812 | the paper, properly credited and are the license and terms of use explicitly mentioned and                                   |
| 813 | properly respected?                                                                                                          |
| 814 | Answer: [N/A]                                                                                                                |
| 815 | Justification: The evaluation does not reuse an external dataset or model; prior methods and                                 |
| 816 | software foundations are credited through citations.                                                                         |
| 817 | <b>13. New assets</b>                                                                                                        |
| 818 | Question: Are new assets introduced in the paper well documented and is the documentation                                    |
| 819 | provided alongside the assets?                                                                                               |



820 Answer: [\[Yes\]](#)

821 Justification: The supplemental artifact documents the checker interface, pack schema,  
822 synthetic corpora, proof scope, limitations, and reproduction workflow.

823 **14. Crowdsourcing and research with human subjects**

824 Question: For crowdsourcing experiments and research with human subjects, does the paper  
825 include the full text of instructions given to participants and screenshots, if applicable, as  
826 well as details about compensation (if any)?

827 Answer: [\[N/A\]](#)

828 Justification: The research involves neither crowdsourcing nor human-subject experiments.

829 **15. Institutional review board (IRB) approvals or equivalent for research with human**  
830 **subjects**

831 Question: Does the paper describe potential risks incurred by study participants, whether  
832 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)  
833 approvals (or an equivalent approval/review based on the requirements of your country or  
834 institution) were obtained?

835 Answer: [\[N/A\]](#)

836 Justification: The research involves no study participants or human-subject data.

837 **16. Declaration of LLM usage**

838 Question: Does the paper describe the usage of LLMs if it is an important, original, or  
839 non-standard component of the core methods in this research? Note that if the LLM is used  
840 only for writing, editing, or formatting purposes and does not impact the core methodology,  
841 scientific rigor, or originality of the research, declaration is not required.

842 Answer: [\[Yes\]](#)

843 Justification: Sections 1 and 4 describe LLM compaction and bounded repair, explicitly  
844 place them outside the trusted checker, and state that the reported corpus evaluation does  
845 not depend on live model inference.